

halfedge：几何图元生成模块设计文档

src/primitives.rs

halfedge 库

2026-07-04

目录

1	模块定位	1
2	参数化公式	2
2.1	立方体	2
2.2	UV 球	2
2.3	圆柱	2
2.4	圆锥	2
2.5	圆环	3
2.6	网格	3
3	参数钳制	3
4	容量预分配	3
5	复杂度	4
6	测试覆盖	4
7	设计取舍	4

1 模块定位

primitives.rs 提供 6 个程序化图元构建器，每个返回完整的 MeshStorage：CCW 朝外法向、容量预分配、调用 add_triangle 自动建立半边拓扑。所有图元位于原点，参数化生成。

图元	API	顶点数	面数
立方体	<code>build_cube(size)</code>	8	12
UV 球	<code>build_uv_sphere(r, slices, stacks)</code>	$(s+1)(t+1)$	$2 \cdot s \cdot t$
圆柱	<code>build_cylinder(r, h, slices)</code>	$2s + 2$	$4s$
圆锥	<code>build_cone(r, h, slices)</code>	$s + 2$	$2s$
圆环	<code>build_torus(R, r, mj, mn)</code>	$mj \cdot mn$	$2 \cdot mj \cdot mn$
网格	<code>build_grid(w, d, sx, sz)</code>	$(s_x+1)(s_z+1)$	$2 \cdot s_x \cdot s_z$

2 参数化公式

2.1 立方体

$h = \text{size}/2$, 8 顶点位于 $(\pm h, \pm h, \pm h)$ 。底面 $-z$ (4 顶点 CCW) + 顶面 $+z$ (4 顶点 CCW), 共 12 三角形 (每面 2 个)。委托 `build_mesh_from_vertices_and_faces` 一次成型。

2.2 UV 球

极角 ϕ 从 $+y$ 极 (0) 到 $-y$ 极 (π), 方位角 $\theta \in [0, 2\pi)$:

$$\phi_i = \frac{\pi \cdot i}{\text{stacks}}, \quad i = 0, \dots, \text{stacks} \quad (1)$$

$$\theta_j = \frac{2\pi \cdot j}{\text{slices}}, \quad j = 0, \dots, \text{slices} \quad (2)$$

$$y_{i,j} = r \cos \phi_i, \quad r_{\text{ring},i} = r \sin \phi_i \quad (3)$$

$$x_{i,j} = r_{\text{ring},i} \cos \theta_j, \quad z_{i,j} = r_{\text{ring},i} \sin \theta_j \quad (4)$$

每个 (i, j) 单元分裂为 2 三角形: $(a, b, c) + (c, b, d)$, 其中 $a = v_{i,j}, b = v_{i,j+1}, c = v_{i+1,j}, d = v_{i+1,j+1}$ 。

缝合警告 列 $j = \text{slices}$ 与 $j = 0$ 是**重复顶点** (位置相同但 ID 不同), 形成拓扑接缝而非闭合环。

2.3 圆柱

沿 y 轴, $y \in [-h/2, +h/2]$ 。两中心顶点 (顶 / 底) + s 个圆环顶点 (顶 / 底各一):

$$\theta_i = \frac{2\pi \cdot i}{\text{slices}}, \quad x_i = r \cos \theta_i, \quad z_i = r \sin \theta_i.$$

每 i 产生 4 三角形: 侧面 2 + 顶帽 1 + 底帽 1, 环用 $(i+1) \bmod s$ 闭合。

2.4 圆锥

顶点 $(0, +h/2, 0)$, 底面圆心 $(0, -h/2, 0)$, 底圆环:

$$\theta_i = \frac{2\pi \cdot i}{\text{slices}}, \quad \text{ring}_i = (r \cos \theta_i, -h/2, r \sin \theta_i).$$

每 i 产生 2 三角形: 侧面 $(\text{apex}, \text{ring}_{i+1}, \text{ring}_i) +$ 底面 $(\text{base}_c, \text{ring}_i, \text{ring}_{i+1})$ 。

2.5 圆环

环面位于 xz 平面，主半径 R （环中心到管中心），副半径 r （管半径）：

$$\theta_i = \frac{2\pi \cdot i}{mj}, \quad i = 0, \dots, mj - 1 \quad (5)$$

$$\phi_j = \frac{2\pi \cdot j}{mn}, \quad j = 0, \dots, mn - 1 \quad (6)$$

$$R'_j = R + r \cos \phi_j \quad (7)$$

$$x_{i,j} = R'_j \cos \theta_i, \quad z_{i,j} = R'_j \sin \theta_i, \quad y_{i,j} = r \sin \phi_j \quad (8)$$

每 (i, j) 单元 2 三角形，索引 $ni = (i + 1) \bmod mj$, $nj = (j + 1) \bmod mn$ 实现环向闭合。

2.6 网格

xz 平面 $y = 0$, $x \in [-w/2, +w/2]$, $z \in [-d/2, +d/2]$, 按 $s_x \times s_z$ 单元细分：

$$x_{ix} = -\frac{w}{2} + \frac{w \cdot ix}{s_x}, \quad z_{iz} = -\frac{d}{2} + \frac{d \cdot iz}{s_z}.$$

每单元 2 三角形： $(a, b, c) + (c, b, d)$ ，与 UV 球相同的分裂模式。

3 参数钳制

所有构建器对参数应用最小值钳制，防止退化：

参数	最小值
<code>slices</code> （球 / 圆柱 / 圆锥）	3
<code>stacks</code> （球）	2
<code>major_segments</code> / <code>minor_segments</code> （环）	3
<code>segments_x</code> / <code>segments_z</code> （网格）	1

4 容量预分配

每个构建器（除 `build_cube` 委托给统一构建器外）入口处调用 `mesh.reserve(v, he, f)`，半边容量按 `faces * 6`（每三角面 3 内部 + 3 潜在边界 twin）预分配，使构建期间 rehash 次数为 0。

5 复杂度

构建器	时间复杂度
<code>build_cube</code>	$O(1)$
<code>build_uv_sphere</code>	$O(\text{slices} \cdot \text{stacks})$
<code>build_cylinder</code>	$O(\text{slices})$
<code>build_cone</code>	$O(\text{slices})$
<code>build_torus</code>	$O(mj \cdot mn)$
<code>build_grid</code>	$O(s_x \cdot s_z)$

所有构建器对输出规模线性，无嵌套搜索。

6 测试覆盖

测试名	验证点
<code>cube_is_closed</code>	<code>build_cube(2.0)</code> : $V = 8$, $F = 12$, 校验通过
<code>uv_sphere_is_closed</code>	<code>build_uv_sphere(1, 16, 8)</code> : $F > 0$, 校验通过
<code>cylinder_counts</code>	<code>build_cylinder(1, 2, 16)</code> : $V = 34$, $F = 64$, 校验通过
<code>torus_counts</code>	<code>build_torus(2, 0.5, 16, 8)</code> : $V = 128$, $F = 256$, 校验通过
<code>cone_counts</code>	<code>build_cone(1, 2, 16)</code> : $V = 18$, $F = 32$, 校验通过
<code>grid_counts</code>	<code>build_grid(2, 2, 4, 4)</code> : $V = 25$, $F = 32$
<code>primitives_pass_validation</code>	6 个图元小参数下 <code>validate_mesh</code> 全通过

全模块共 7 个单元测试，全库共 371 个。

7 设计取舍

- 为何 `add_triangle` 的 `Result` 被忽略？参数化构建器通过索引算术保证拓扑合法，错误不应发生。若发生（索引 bug），测试中的 `validate_mesh` 会暴露。生产代码可改为 `expect`。
- 为何 UV 球保留接缝重复顶点？简化索引生成（无需模运算闭合），代价是接缝处多一系列顶点。如需拓扑闭合，应使用 `build_torus` 风格的 $(i + 1) \bmod n$ 索引。
- 为何参数钳制而非返回 `Result`？钳制给调用方零开销的「安全默认」，避免 API 复杂化。调用方仍可传入大值得到大网格，不会失败。